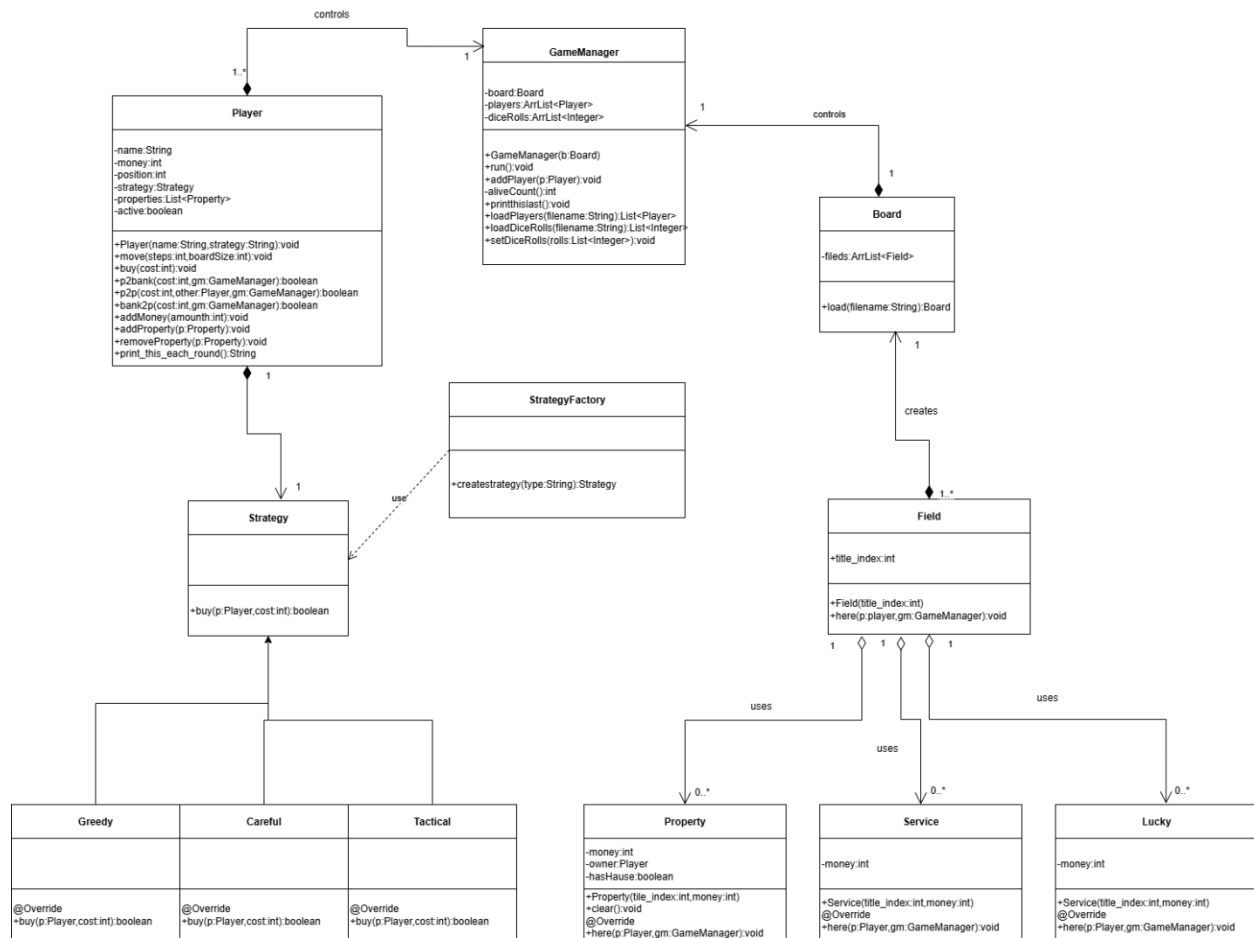


Umut Arda Korkmaz (PG2QXR) Simulate a simplified Capital game. There are some players with different strategies, and a cyclical board with several fields. Players can move around the board, by moving forward with the amount they rolled with a dice. A field can be a property, service, or lucky field. A property can be bought for 1000, and stepping on it the next time the player can build a house on it for 4000. If a player steps on a property field which is owned by somebody else, the player should pay to the owner 500, if there is no house on the field, or 2000, if there is a house on it. Stepping on a service field, the player should pay to the bank (the amount of money is a parameter of the field). Stepping on a lucky field, the player gets some money (the amount is defined as a parameter of the field). There are three different kind of strategies exist. Initially, every player has 10000. Greedy player: If he steps on an unowned property, or his own property without a house, he starts buying it, if he has enough money for it. Careful player: he buys in a round only for at most half the amount of his money. Tactical player: he skips each second chance when he could buy. If a player has to pay, but he runs out of money because of this, he loses. In this case, his properties are lost, and become free to buy. Read the parameters of the game from a text file. This file defines the number of fields, and then defines them. We know about all fields: the type. If a field is a service or lucky field, the cost of it is also defined. After the these parameters, the file tells the number of the players, and then enumerates the players with their names and strategies. In order to prepare the program for testing, make it possible to the program to read the roll dices from the file. Print out what we know about each player after a given number of rounds (balance, owned properties)



GameManager;

addPlayer(Player p)-adds a player to the game

run()-executes the game loop

aliveCount()-returns the number of active players

print_this_last()-prints at the end for end of game stats

loadPlayers(String filename)-reads player names and strategies from txt file

loadDiceRolls(String filename)- reads dice rolls from txt file

Player:

move(int steps, int boardSize)-makes the player able to move in board

buy(int cost)-checks player strategy and deals with buying stuff

p2bank(int cost, GameManager gm)-from player to bank(nowhere just a placeholder we

just subtract that money) money transfer, handles player loosing

bank2p(int cost, GameManager gm)-adds money to player balance

addMoney(int amount)-another way to add money to player balance

addProperty(Property p)-adds property to player

removeProperty(Property p)-removes property from player

print_this_each_round()-return string and we print this each round

Strategy(abstract class) and her subclasses&StrategyFactory:

createStrategy(String type)-deals with strategy creation, can throw exception

Strategy.buy(Player p, int cost)-abstract

Greedy.buy (Player p, int cost)-buys all they can

Careful.buy(Player p, int cost)-buys if in that round its half their money

Tactical.buy(Player p, int cost)-buys every other opportunity by Boolean switching

Board:

size()-return number of fields on board

load(String filename)-loads fields onto board

Field:

here(Player p, GameManager gm)-abstract

Servive&Lucky:

Service. here(Player p, GameManager gm)-player pays money to bank as stated in service class

Lucky. here(Player p, GameManager gm)-player gets paid money as stated in lucky class

Property:

clear()-sets owner of property to null

here(Player p, GameManager gm)-handles buy and place building, player can loose the game in this class, throws exception if something really dumb happens here since this method is one of the main methods of the program.

To test the program we need

-data.txt

-dice_roll.txt

-players.txt

.

So to check different inputs pick a set from one of the case folders case 0 is for default inputs which are already in use.

In each folder after name the condition checked is written

case(index)-(condition) --> folder naming format

So what we give and what we expect;

0-default variant that is already used, expecting the game to end as dice rolls end.

1-dice input wrong, game should never start and should end similar to case 0 but it does not throw an exception just console message since some wrong inputs won't break the game yet.

2-wrong size given as input, we expect the game to run smoothly since we don't read size we skip it and when we need the size of them we just check size and get the value from there.

3-game that should end by last one standing, the case where we force last one standing condition.

4-long game that takes a lot of turns, to check what happens if we have more dice rolls than board size and check what could happen in a really long game.

	What we give	What we expect	What we get	What to test
Case0	Case where dice rolls ends	Game to end with a winner	Game over Final state: Player: ALEX Money: 7000 Properties: 2, Player: MARK Money: 10600 Properties: None Player: CELINE Money: 7000 Properties: 3,	What happens when dice rolls ends
Case1	wrong formatted dice rolls	Game to read them as invalid input	Invalid dice roll: a Invalid dice roll: b Invalid dice roll: c Invalid dice roll: d Invalid dice roll: e Invalid dice roll: f Invalid dice roll: g Invalid dice roll: h Game over Final state: Player: ALEX Money: 10000 Properties: None Player: MARK Money: 10000 Properties: None Player: CELINE Money: 10000 Properties: None	Wrong input format
Case2	Wrong size of values given	To skip the size and get the size from input len	Game over Final state: Player: ALEX Money: 9000 Properties: 2, 0, Player: MARK Money: 8500 Properties: None Player: CELINE Money: 2900 Properties: 3,	What code does when size is wrong
Case3	Long inout for dice rools	Game ends by last one standing	Game over Final state: Player: ALEX Money: 23200 Properties: 0, MARK has LOST CELINE has LOST	What happens when one player stands last
Case4	Input as where no win condition	Game to end with dice rolls end	Game over Final state: Player: ALEX Money: 306000 Properties: None Player: MARK Money: 305000 Properties: None Player: CELINE Money: 305000 Properties: None	What happens when there is no win condition